

Teach Module 44: Continuous Delivery and Deployment Pipelines

This note is based on Module 44's topic title from the bootcamp outline, but the teaching content is original and does not use the course material as the lesson source.

Module 44 Overview

Module 44 covers **Continuous Delivery and Deployment Pipelines**.

The main idea is this:

- **Continuous Integration (CI)** asks: "Does the code build and pass tests?"
- **Continuous Delivery (CD)** asks: "Can this tested code safely move toward users?"

Continuous Delivery and Deployment are about taking a validated application artifact and moving it through environments like `dev` , `staging` , and `production` in a repeatable, automated, low-risk way.

CI vs CD

Continuous Integration usually includes:

```
Developer commits code
    ↓
Build runs
    ↓
Tests run
    ↓
Security/static checks run
    ↓
Artifact is produced
```

Example artifacts:

- `.jar` file
- Docker image
- `.war` file
- frontend build folder
- versioned package

Continuous Delivery starts after that:

```
Tested artifact
    ↓
Deploy to staging
    ↓
Run smoke tests
    ↓
Wait for approval
```

↓
Deploy to production

The key idea is: **build once, then promote the same artifact forward.**

Avoid this:

Build for dev
Build again for staging
Build again for production

Prefer this:

Build once
Deploy same artifact to dev
Promote same artifact to staging
Promote same artifact to production

This prevents the problem where staging tested one build, but production received a different one.

Continuous Delivery vs Continuous Deployment

Continuous Delivery means the system can deploy to production at any time, but a human may approve the final step.

Code merged
Build and tests pass
Deploy to staging
Manual approval
Deploy to production

Continuous Deployment means every change that passes the pipeline automatically goes to production.

Code merged
Build and tests pass
Deploy to production automatically

Many enterprise teams begin with Continuous Delivery because it combines automation with human control.

Common Deployment Pipeline Stages

A typical deployment pipeline may look like this:

Source
↓
Build
↓
Unit Tests
↓

```
Package Artifact
↓
Deploy to Test
↓
Integration Tests
↓
Deploy to Staging
↓
Approval Gate
↓
Deploy to Production
↓
Post-deployment Checks
```

Each stage answers a question:

- `Build` : Can the application compile/package?
- `Unit Tests` : Do small pieces of logic work?
- `Integration Tests` : Does the app work with databases, APIs, queues, and other systems?
- `Security Scan` : Are there dependency or code vulnerabilities?
- `Deploy to Staging` : Can this artifact run in a production-like environment?
- `Approval Gate` : Is the team or business ready to release?
- `Production Deploy` : Can users receive the new version?
- `Post-deployment Checks` : Is the system healthy after release?

Environment Promotion

Environment promotion means moving the same version through increasingly serious environments.

```
v1.8.2 → dev → test → staging → production
```

Each environment has a purpose:

- `dev` : Fast feedback for developers.
- `test` : Automated testing environment.
- `staging` : Production-like validation.
- `production` : Real users.

The important rule:

Configuration changes by environment, but the application artifact should not.

For example, the app version stays the same:

```
app: orders-service:1.8.2
```

But configuration changes:

```
DEV_DATABASE_URL
STAGING_DATABASE_URL
PROD_DATABASE_URL
```

Approval Gates

An approval gate is a pause before an important action.

Example:

```
Deploy to staging
Run smoke tests
Wait for release manager approval
Deploy to production
```

Approval gates are useful when:

- production releases need business sign-off
- database migrations are involved
- deployments happen during a release window
- risk is high
- compliance or audit trails are required

Approval gates should not replace automated checks. They should sit on top of them.

```
Automation proves the release is technically ready.
Approval confirms the organization is ready.
```

Rollback and Recovery

A CD pipeline is incomplete if it only knows how to deploy forward. It also needs a recovery plan.

Common recovery strategies:

Rollback

Return to the previous known-good version.

```
production: v1.8.3 has issue
rollback to v1.8.2
```

Roll Forward

Fix the bug and deploy a new version.

```
production: v1.8.3 has issue
deploy v1.8.4 with fix
```

Blue-Green Deployment

Run two production environments:

```
Blue: current production
Green: new version
```

Traffic switches from Blue to Green only when Green is healthy. If Green fails, traffic goes back to Blue.

Canary Deployment

Release to a small percentage of users first.

```
5% users → new version
25% users → new version
100% users → new version
```

If errors increase, stop the rollout.

Terraform and Ansible in CD

Terraform is commonly used for infrastructure provisioning. It creates or updates things like:

- cloud servers
- networks
- databases
- Kubernetes clusters
- load balancers
- storage buckets

Example pipeline use:

```
terraform init
terraform plan
terraform apply
```

Terraform is best at answering:

```
What infrastructure should exist?
```

Ansible is commonly used for configuration and deployment tasks. It can:

- install packages
- copy files
- configure services
- restart applications
- apply server settings
- run deployment scripts

Ansible is best at answering:

```
How should these machines be configured?
```

Together:

```
Terraform creates the server
Ansible configures the server
```

Pipeline deploys the application

Example Java Application CD Flow

For a Java Spring Boot app, a CD pipeline might look like this:

```
Developer merges code to main
↓
Maven builds the app
↓
Tests run
↓
Docker image is created
↓
Image is pushed to registry
↓
Deploy image to staging
↓
Run smoke tests
↓
Manual approval
↓
Deploy same image to production
↓
Monitor logs, metrics, and health checks
```

Example artifact:

```
registry.example.com/orders-service:1.4.0
```

That exact same image should go to staging and production.

Practice Exercises

Exercise 1: Draw a CD Pipeline

Create a pipeline diagram for a Java Spring Boot app:

```
Commit
Build
Unit Tests
Package JAR
Build Docker Image
Deploy to Staging
Smoke Tests
Approval
Deploy to Production
Health Check
Rollback
```

For each stage, write:

What happens here?
What tool could perform this step?
What should happen if this step fails?

Exercise 2: CI vs CD Sorting Activity

Sort these tasks into **CI** or **CD**:

Compile Java code
Run unit tests
Deploy to staging
Run smoke tests
Create Docker image
Manual production approval
Deploy to production
Rollback to previous version
Run dependency scan
Promote artifact

Then explain why each belongs there.

Exercise 3: Build Once, Deploy Many

Imagine your pipeline creates this artifact:

orders-service:1.2.0

Write how that same artifact moves through:

dev
test
staging
production

Then explain why rebuilding separately for production is risky.

Exercise 4: Deployment Strategy Comparison

Make a table comparing:

Rolling Deployment
Blue-Green Deployment
Canary Deployment
Manual Deployment

Use these columns:

How it works
Advantages
Risks
Best use case
Rollback approach

Exercise 5: Design a Rollback Plan

Scenario:

Version 2.0.0 was deployed to production.
Users report checkout failures.
Logs show payment API calls are failing.

Write a rollback plan:

How do you detect the failure?
Who approves rollback?
What version do you roll back to?
How do you confirm recovery?
What logs or metrics do you check after rollback?

Exercise 6: Staging Approval Gate

Design an approval process before production deployment.

Include:

Who approves?
What checks must pass first?
What evidence should reviewers see?
What happens if approval is denied?
What happens if approval is granted?

Exercise 7: CD Pipeline YAML Practice

Write pseudocode for a pipeline like this:

```
stages:  
  - build  
  - test  
  - package  
  - deploy_staging  
  - smoke_test  
  - approval  
  - deploy_production
```

For each stage, add two or three commands or comments describing what it does.

Exercise 8: Terraform and Ansible CD Workflow

Design a pipeline where:

```
Terraform provisions infrastructure
Ansible configures the server
The Java app is deployed
Smoke tests verify deployment
```

Write the flow like this:

```
Step:
Tool:
Purpose:
Failure handling:
```

Exercise 9: Secrets and Environment Variables

List the configuration values a Java app might need in different environments:

```
Database URL
Database username
Database password
JWT secret
Payment API key
Logging level
Feature flags
```

Then classify each one:

```
Can be plain config
Must be secret
Environment-specific
Same across all environments
```

Exercise 10: Production Deployment Readiness Checklist

Create a checklist that must be completed before production deployment.

Include items for:

```
Testing
Security
Database migration
Rollback
Monitoring
Approval
Communication
Post-deployment validation
```

Example:

```
[ ] Artifact version confirmed
[ ] Staging deployment successful
[ ] Smoke tests passed
[ ] Rollback version identified
[ ] Production health checks configured
```

Lab: Continuous Delivery Pipeline

Lab Goal

Build a simple CD pipeline for a Java Spring Boot application that:

```
Builds the app
Runs tests
Creates a deployable artifact
Deploys to staging
Runs a smoke test
Requires approval
Deploys to production
Includes rollback steps
```

Lab Scenario

You have a Spring Boot app called:

```
orders-service
```

It exposes:

```
GET /actuator/health
GET /api/orders
```

You need to create a delivery pipeline for versioned deployment.

Step 1: Prepare the Application

Make sure the app can be built with Maven:

```
mvn clean package
```

Expected result:

```
target/orders-service-1.0.0.jar
```

If you do not already have a Spring Boot app, use any simple Java app and treat the `.jar` as the deployable artifact.

Step 2: Create Pipeline Stages

Design these stages:

```
build
test
package
deploy_staging
smoke_test_staging
approval
deploy_production
smoke_test_production
rollback
```

Step 3: Build Stage

Purpose: compile the Java application.

Example command:

```
mvn clean compile
```

Failure rule:

```
If compilation fails, stop the pipeline.
```

Step 4: Test Stage

Purpose: run automated tests.

Example command:

```
mvn test
```

Failure rule:

```
If any test fails, do not package or deploy.
```

Step 5: Package Stage

Purpose: create the final deployable `.jar`.

Example command:

```
mvn clean package -DskipTests
```

Expected artifact:

```
target/orders-service-1.0.0.jar
```

Record the artifact version:

```
orders-service version: 1.0.0  
build number: 44-lab-001
```

Step 6: Create a Docker Image

Create a simple Dockerfile :

```
FROM eclipse-temurin:17-jre  
WORKDIR /app  
COPY target/*.jar app.jar  
EXPOSE 8080  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build the image:

```
docker build -t orders-service:1.0.0 .
```

Step 7: Deploy to Staging

Run the app as a staging container:

```
docker run -d --name orders-service-staging -p 8081:8080 orders-service:1.0.0
```

Staging URL:

```
http://localhost:8081
```

Step 8: Run Staging Smoke Test

Check the health endpoint:

```
curl http://localhost:8081/actuator/health
```

Expected result:

```
{"status":"UP"}
```

If the smoke test fails:

```
Stop the pipeline.  
Do not deploy to production.  
Check container logs.
```

Check logs:

```
docker logs orders-service-staging
```

Step 9: Approval Gate

Before production deployment, answer these:

```
Did the build pass?  
Did tests pass?  
Did staging deploy successfully?  
Did smoke tests pass?  
Is rollback version known?  
Is production deployment approved?
```

Write approval like this:

```
Approved by:  
Date:  
Version approved:  
Rollback version:
```

Step 10: Deploy to Production

Run production container:

```
docker run -d --name orders-service-prod -p 8080:8080 orders-service:1.0.0
```

Production URL:

```
http://localhost:8080
```

Step 11: Run Production Smoke Test

```
curl http://localhost:8080/actuator/health
```

Expected result:

```
{"status":"UP"}
```

Also test one business endpoint:

```
curl http://localhost:8080/api/orders
```

Step 12: Rollback Practice

Simulate a bad deployment by trying a newer version:

```
docker stop orders-service-prod
docker rm orders-service-prod
docker run -d --name orders-service-prod -p 8080:8080 orders-service:1.1.0
```

Assume version 1.1.0 fails.

Rollback to 1.0.0 :

```
docker stop orders-service-prod
docker rm orders-service-prod
docker run -d --name orders-service-prod -p 8080:8080 orders-service:1.0.0
```

Verify:

```
curl http://localhost:8080/actuator/health
```

Lab Deliverables

Create a short lab report with:

```
Application name:
Artifact version:
Pipeline stages:
Staging URL:
Production URL:
Smoke test result:
Approval decision:
Rollback version:
Rollback command:
Lessons learned:
```

Challenge Version

After completing the manual version, automate it with a pipeline YAML.

Example structure:

```
stages:
- build
- test
- package
- deploy_staging
- smoke_test
- approval
- deploy_production
```

The main learning outcome is to understand how a tested Java artifact moves safely from build to staging to production, with approval and rollback built into the process.